Лекция 2. Автоматы: бор, Ахо-Корасик.

1. Введение в автоматы

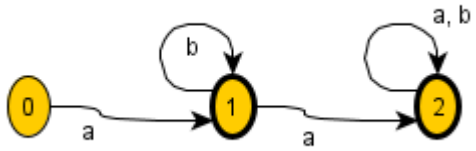
Автомат – это ориентированный граф, представляющий собой модель вычислений, которая обрабатывает строку символ за символом. Автомат используется для распознавания строк, принадлежащих определённому языку (множеству строк).

Определение:

- Автомат задаётся набором состояний (вершин графа), одним начальным состоянием, набором терминальных (финальных) состояний и функцией переходов по символам алфавита.
- Автомат начинает работу в начальном состоянии.
- На каждом шаге он читает очередной символ строки и совершает переход в следующее состояние в соответствии с функцией переходов.
- Если после обработки всей строки автомат оказывается в терминальном состоянии, строка **принимается** (принадлежит языку). Иначе – отвергается.

Пример простого автомата:

Автомат, принимающий все строки, начинающиеся с символа 'a', за которым следует произвольное количество символов 'b', а затем символ 'a'.



Такой автомат принимает строки: "aba", "abbbba", "aa" и т.д.

2. Бор (префиксное дерево)

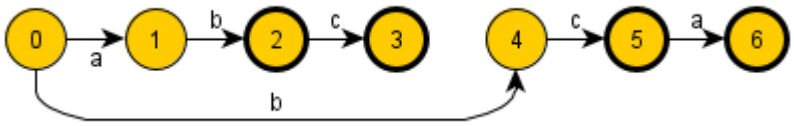
Бор – это дерево, предназначенное для хранения набора строк. Каждая вершина соответствует некоторому префиксу одной или нескольких строк из набора. Переходы помечены символами алфавита.

Построение бора:

- Начинаем с корня (пустая строка).
- Для каждой строки в наборе начинаем с корня и посимвольно добавляем переходы, создавая новые вершины при необходимости.
- В терминальных вершинах помечаем, что в этой вершине заканчивается одна из строк набора.

Пример:

Набор строк: {"ab", "abc", "bc", "bca"}.



Псевдокод построения бора:

```
class TrieNode:
   children = {}  # словарь: символ -> узел
   # Ключ - символ алфавита, значение - ссылка на следующий узел
   # Это позволяет хранить переходы по конкретным символам

   is_terminal = False
   # Флаг, указывающий, является ли текущая вершина концом какой-либо строки
   # Если True, то от корня до этой вершины - одна из строк нашего набора

   # Дополнительные поля, которые могут понадобиться для Ахо-Корасик:
   # parent = None          # ссылка на родительский узел
   # suffix_link = None     # суффиксная ссылка
   # output_link = None     # ссылка на ближайшую терминальную вершину
   # depth = 0              # глубина вершины (длина префикса)

function build_trie(strings):
   # Создаём корень бора - соответствует пустой строке
   root = TrieNode()

   # Перебираем все строки из набора
   for s in strings:
      # Начинаем с корня для каждой новой строки
      node = root

      # Обрабатываем строку посимвольно
      for char in s:
         # Если из текущего узла нет перехода по текущему символу
         if char not in node.children:
            # Создаём новый узел для этого символа
            node.children[char] = TrieNode()

         # Переходим в узел, соответствующий текущему символу
         node = node.children[char]

      # После обработки всех символов строки помечаем конечный узел как терминальный
      # Это означает, что строка s присутствует в боре
      node.is_terminal = True
```

```
# Возвращаем корень построенного бора
return root
```

Применение бора:

- Быстрая проверка наличия строки в наборе – $O(|s|)$.
- Поиск всех строк с заданным префиксом.
- Основной строительный блок для более сложных структур (автомат Ахо-Корасик).

3. Автомат Ахо-Корасик

Автомат Ахо-Корасик расширяет бор, добавляя **суффиксные ссылки**, что позволяет эффективно искать все вхождения множества строк в тексте без возвратов.

Идея:

- Построим бор для набора искомых строк.
- Для каждой вершины бора определим **суффиксную ссылку** – указатель на вершину, соответствующую наибольшему собственному суффиксу текущего префикса, который также является префиксом какой-либо строки из набора.
- Добавим **переходы** (иногда называемые “сжатыми” переходами), которые позволяют за $O(1)$ переходить из текущей вершины по любому символу, даже если непосредственного перехода нет.

Суффиксные ссылки:

- Суффиксная ссылка из вершины v ведёт в вершину u, такую что строка, соответствующая u, является наибольшим суффиксом строки v, присутствующим в боре.
- Суффиксная ссылка из корня ведёт в корень.
- С помощью суффиксных ссылок можно быстро “перескакивать” при несовпадениях, аналогично префикс-функции в КМП.

Построение автомата (BFS):

1. Построить бор.
2. Проинициализировать суффиксные ссылки для детей корня (ведут в корень).
3. Обработать остальные вершины в порядке BFS:
   - Суффиксная ссылка для вершины v по символу c вычисляется через суффиксную ссылку родителя v.
4. Одновременно вычислять переходы: если из текущей вершины нет перехода по символу c, то переход равен переходу по суффиксной ссылке.

Псевдокод построения автомата Ахо-Корасик:

```
class AhoNode:
  children = {}      # непосредственные дети в боре (только явные переходы)
  go = {}            # полная таблица переходов (включая сжатые через суффиксные ссылки)
```

```
                # go[символ] даёт следующий узел автомата при чтении этого символа

    parent = None        # ссылка на родительский узел (нужна для вычисления суффиксных ссылок)
    char_from_parent = None  # символ, по которому перешли от родителя к этому узлу

    suffix_link = None # суффиксная ссылка - аналог значения префикс-функции
                # указывает на узел, соответствующий наибольшему собственному суффиксу

    output_link = None # выходная ссылка - указатель на ближайшую терминальную вершину
                # в цепочке суффиксных ссылок (или None, если таковой нет)

    is_terminal = False # флаг терминальности вершины
    matched_string = None # строка, которая заканчивается в этой вершине (опционально)

function build_aho_corasick(strings):
    # Шаг 1: Создаём корень автомата
    root = AhoNode()

    # Шаг 2: Построение бора (первоначальная структура)
    for s in strings:
        node = root
        for char in s:
            if char not in node.children:
                # Создаём новый узел
                new_node = AhoNode()
                new_node.parent = node
                new_node.char_from_parent = char
                node.children[char] = new_node
            node = node.children[char]
        # Помечаем конечную вершину как терминальную
        node.is_terminal = True
        # Сохраняем строку, которая соответствует этой вершине
        node.matched_string = s

    # Шаг 3: BFS-обход для вычисления суффиксных ссылок и переходов
    queue = []  # очередь для обхода в ширину
```

```
    # Инициализация для непосредственных детей корня
    for char, child in root.children.items():
        # Суффиксная ссылка детей корня ведёт в корень
        child.suffix_link = root
        # Добавляем детей в очередь для дальнейшей обработки
        queue.append(child)

    # Основной цикл BFS
    while queue:
        node = queue.pop(0)  # извлекаем узел из начала очереди

        # Вычисляем переходы (go) для всех символов алфавита
        for char in alphabet:
            if char in node.children:
                # Если есть прямой переход по символу, используем его
                node.go[char] = node.children[char]
            elif node == root:
                # Если узел - корень и нет перехода, остаёмся в корне
                node.go[char] = root
            else:
                # Иначе переходим по суффиксной ссылке и проверяем переход
оттуда
                # Это рекурсивный вызов, но так как мы идём BFS, нужные
суффиксные ссылки уже вычислены
                node.go[char] = node.suffix_link.go[char]

        # Вычисляем суффиксные ссылки для детей текущего узла
        for char, child in node.children.items():
            # Суффиксная ссылка для ребёнка вычисляется через суффиксную
ссылку родителя
            if node == root:
                # Если родитель - корень, суффиксная ссылка ребёнка ведёт в корень
                child.suffix_link = root
            else:
                # Иначе: идём по суффиксной ссылке родителя и смотрим,
                # есть ли оттуда переход по текущему символу
                child.suffix_link = node.suffix_link.go[char]

            # Вычисляем выходную ссылку (output link)
            # Если суффиксная ссылка ведёт в терминальную вершину, запоминаем
её
            if child.suffix_link.is_terminal:
                child.output_link = child.suffix_link
```

```
        else:
            # Иначе берём выходную ссылку от суффиксной ссылки
            child.output_link = child.suffix_link.output_link

        # Добавляем ребёнка в очередь для обработки
        queue.append(child)

    return root
```

4. Поиск строк с помощью автомата Ахо-Корасик

Алгоритм поиска:

1. Начинаем в корне автомата.
2. Посимвольно обрабатываем текст.
3. Для каждого символа совершаем переход с использованием предвычисленных переходов (go).
4. В каждой достигнутой вершине проверяем:

- Является ли она терминальной – найдено вхождение строки, соответствующей этой вершине.
- Поднимаемся по output-ссылкам, чтобы найти все вхождения, которые заканчиваются в данной позиции текста.

Псевдокод поиска:

```
function aho_corasick_search(text, strings):
    # Шаг 1: Строим автомат Ахо-Корасик для заданного набора строк
    root = build_aho_corasick(strings)

    # Шаг 2: Инициализируем начальное состояние - корень автомата
    node = root

    # Шаг 3: Подготавливаем структуру для результатов
    # Ключ - строка, значение - список позиций её вхождений в текст
    results = {s: [] for s in strings}

    # Шаг 4: Посимвольный проход по тексту
    for i in range(len(text)):
        char = text[i]  # текущий символ текста

        # Совершаем переход по текущему символу
        # Используем предвычисленную таблицу переходов go
        node = node.go[char]

        # Проверяем, является ли текущая вершина терминальной
        if node.is_terminal:
            # Нашли строку, соответствующую текущей вершине
            # Вычисляем позицию начала строки в тексте
```

```
        start_pos = i - len(node.matched_string) + 1
        results[node.matched_string].append(start_pos)

    # Проверяем выходные ссылки (output links)
    # Они позволяют найти все строки, которые заканчиваются в текущей позиции,
    # но соответствуют более коротким суффиксам
    output_node = node.output_link
    while output_node is not None:
        # Добавляем вхождение строки, соответствующей output_node
        start_pos = i - len(output_node.matched_string) + 1
        results[output_node.matched_string].append(start_pos)
        # Переходим к следующей выходной ссылке в цепочке
        output_node = output_node.output_link

# Возвращаем все найденные вхождения
return results
```

Сложность:

- Построение автомата: $O(\Sigma \cdot L)$, где L – суммарная длина всех строк, Σ – размер алфавита.
- Поиск: $O(n + m + k)$, где n – длина текста, m – суммарная длина строк, k – общее количество вхождений.

5. Пример работы

Пусть набор строк: {"he", "she", "his", "hers"}.

Текст: "ushers".

1. Построим бор и автомат.
2. Обрабатываем текст:

Символ 'u' – остаёмся в корне.

's' – переходим в вершину "s".

'h' – переходим в "sh".

'e' – переходим в "she" (терминал, найдено "she" на позиции 1).

Также по output-ссылке переходим к "he" (терминал, найдено "he" на позиции 2).

'r' – переходим в "her".

's' – переходим в "hers" (терминал, найдено "hers" на позиции 2).

Результат:

- "she" → [1]
- "he" → [2]
- "hers" → [2]

6. Сравнение с другими методами

Метод	Время (препроцессинг + поиск)	Память	Особенности
Наивный поиск	$O(n \cdot m)$	$O(1)$	Просто, но медленно
КМР (одна строка)	$O(m + n)$	$O(m)$	Надёжный, без возвратов
Рабин-Карп	$O(m + n)$ в среднем	$O(1)$	Вероятностный, быстро на практике
Бор	$O(\Sigma \cdot L)$	$O(\Sigma \cdot L)$	Быстрая проверка наличия строк
Ахо-Корасик	$O(\Sigma \cdot L + n + k)$	$O(\Sigma \cdot L)$	Поиск множества строк в тексте, детерминированный

7. Дополнительные темы

7.1 Минимизация автомата

Минимизация автомата – это процесс построения эквивалентного автомата с минимальным количеством состояний. Для детерминированных конечных автоматов (ДКА) существует алгоритм минимизации за $O(n \cdot \log n)$.

В контексте Ахо-Корасик минимизация может быть полезна для уменьшения памяти, но на практике часто используют не минимальный автомат из-за сложности минимизации при динамическом добавлении строк.

7.2 Проверка эквивалентности автоматов

Два автомата эквивалентны, если они принимают один и тот же язык.

Проверка эквивалентности может быть выполнена путём одновременного обхода обоих автоматов и проверки совпадения терминальности состояний.

7.3 Оптимизации для Ахо-Корасик

- **Сжатие переходов:** вместо массива переходов размера Σ можно хранить переходы в хеш-таблице или списке.
- **Ленивое вычисление переходов:** вычислять переходы только при необходимости, экономя память.
- **Использование output-ссылок:** позволяет быстро находить все вхождения без повторного подъёма по суффиксным ссылкам.